

Pseudocode codingan

constant NMAX : integer = 100

constant MAXMK : integer = 20

type Mahasiswa <

 NIM : string

 Nama : string

>

type NilaiMK <

 Kode : string

 Nama : string

 Grade : string

 SKS : integer

 Quiz : real

 UTS : real

 UAS : real

 Total : real

>

type KRS <

 NIM : string

 Jml : integer

 MK : array [0..MAXMK-1] of NilaiMK

>

type arrMhs : array [0..NMAX-1] of Mahasiswa

type arrKRS : array [0..NMAX-1] of KRS

function SeqSearch(T : arrMhs, n : integer, X : string) → integer

Kamus

i, idx : integer

Algoritma

idx ← -1

i ← 0

while i < n and idx = -1 do

 if T[i].NIM = X then

 idx ← i

 endif

 i ← i + 1

endwhile

return idx

endfunction

function BinarySearch(A : arrMhs, n : integer, x : string) → integer

Kamus

left, right, mid : integer

Algoritma

left ← 0

right ← n – 1

while left ≤ right do

 mid ← (left + right) / 2

 if A[mid].NIM = x then

 return mid

 else if x < A[mid].NIM then

```

        right ← mid - 1
    else
        left ← mid + 1
    endif
endwhile
return -1
endfunction

```

function totalNilai(quiz, uts, uas : real) → real

Kamus

```
total : real
```

Algoritma

```

    total ← (0.2 × quiz) + (0.3 × uts) + (0.5 × uas)
    return total
endfunction

```

function grade(total : real) → string

Algoritma

```

    if total ≥ 80 then
        return "A"
    else if total ≥ 75 then
        return "AB"
    else if total ≥ 70 then
        return "B"
    else if total ≥ 65 then
        return "BC"
    else if total ≥ 60 then

```

```

        return "C"
    else if total  $\geq$  50 then
        return "D"
    else
        return "E"
    endif
endfunction

```

function totalSKS(data : KRS) $\rightarrow$ integer

Kamus

```

    i : integer
    total : integer

```

Algoritma

```

    total  $\leftarrow$  0
    for i  $\leftarrow$  0 to data.Jml - 1 do
        total  $\leftarrow$  total + data.MK[i].SKS
    endfor
    return total
endfunction

```

function rataNilai(data : KRS) $\rightarrow$ real

Kamus

```

    i : integer
    total : real

```

Algoritma

```

    if data.Jml = 0 then

```

```

        return 0
    endif
    total ← 0
    for i ← 0 to data.Jml - 1 do
        total ← total + data.MK[i].Total
    endfor
    return total / data.Jml
endfunction

```

procedure tambahMahasiswa(in/out A : arrMhs, in/out B : arrKRS, in/out n : integer)

Algoritma

```

    if n ≥ NMAX then
        output("Data mahasiswa penuh")
    else
        input(A[n].NIM)
        input(A[n].Nama)
        B[n].NIM ← A[n].NIM
        B[n].Jml ← 0
        n ← n + 1
        output("Data mahasiswa berhasil ditambahkan")
    endif
endprocedure

```

procedure editMahasiswa(in/out A : arrMhs, in n : integer)

Kamus

```

    nim : string
    idx : integer

```

Algoritma

```

    Output("Masukkan NIM yang akan diedit : ")
    input(nim)

```

```

idx ← BinarySearch(A, n, nim)
if idx = -1 then
    output("Mahasiswa tidak ditemukan")
else
    output("Nama baru : ")
    input(A[idx].Nama)
    output("Data mahasiswa berhasil diubah")
endif
endprocedure

```

procedure hapusMahasiswa(in/out A : arrMhs, in/out B : arrKRS, in/out n : integer)

Kamus

nim : string

idx : integer

i : integer

Algoritma

Output("Masukkan NIM yang akan dihapus :")

input(nim)

idx ← BinarySearch(A, n, nim)

if idx = -1 then

output("Mahasiswa tidak ditemukan")

else

for i ← idx to n - 2 do

A[i] ← A[i+1]

B[i] ← B[i+1]

Endfor

n ← n - 1

output("Data mahasiswa berhasil dihapus")

```
endif  
endprocedure
```

procedure tampilSemuaMahasiswa(in A : arrMhs, in B : arrKRS, in n : integer)

Kamus

i : integer

Algoritma

```
if n = 0 then  
    output("Belum ada data mahasiswa")  
else  
    output(header tabel: NIM, Nama, Total SKS, Rata-rata)  
    for i ← 0 to n - 1 do  
        output(A[i].NIM, A[i].Nama, totalSKS(B[i]), rataNilai(B[i]))  
    endfor  
endif  
endprocedure
```

procedure tambahMK(in A : arrMhs, in/out B : arrKRS, in n : integer)

Kamus

nim : string

idx : integer

j : integer

Algoritma

```
input(nim)  
idx ← SeqSearch(A, n, nim)  
if idx = -1 then  
    output("Mahasiswa tidak ditemukan")  
else
```

```

    j ← B[idx].Jml
    if j ≥ MAXMK then
        output("Mata kuliah sudah penuh")
    else
        input(B[idx].MK[j].Kode)
        input(B[idx].MK[j].Nama)
        input(B[idx].MK[j].SKS)

        input(B[idx].MK[j].Quiz)
        input(B[idx].MK[j].UTS)
        input(B[idx].MK[j].UAS)
        B[idx].MK[j].Total ← totalNilai(B[idx].MK[j].Quiz, B[idx].MK[j].UTS,
        B[idx].MK[j].UAS)
        B[idx].MK[j].Grade ← grade(B[idx].MK[j].Total)
        B[idx].Jml ← B[idx].Jml + 1
        output("Mata kuliah berhasil ditambahkan")
    endif
endif
endprocedure

```

procedure editMK(in A : arrMhs, in/out B : arrKRS, in n : integer)

Kamus

nim : string

kode : string

idx : integer

j : integer

ketemu : Boolean

Algoritma

Output(" Masukkan NIM : ")


```

        input(nim)
    idx ← SeqSearch(A, n, nim)
    if idx = -1 then
        output("Mahasiswa tidak ditemukan")
    else
        input(kode)
        ketemu ← false
        j ← 0

        while j < B[idx].Jml and not ketemu do
            if B[idx].MK[j].Kode = kode then
                input(B[idx].MK[j].Quiz)
                input(B[idx].MK[j].UTS)
                input(B[idx].MK[j].UAS)
                B[idx].MK[j].Total ← totalNilai(B[idx].MK[j].Quiz,
                B[idx].MK[j].UTS, B[idx].MK[j].UAS)
                B[idx].MK[j].Grade ← grade(B[idx].MK[j].Total)
                output("Data mata kuliah berhasil diubah")
                ketemu ← true
            endif
            j ← j + 1
        endwhile

        if not ketemu then
            output("Mata kuliah tidak ditemukan")
        endif
    endif
endprocedure

```

procedure hapusMK(in A : arrMhs, in/out B : arrKRS, in n : integer)

Kamus

nim : string

kode : string

idx : integer

j : integer

k : integer

pos : integer

Algoritma

Output("Masukkan NIM : ")

idx ← SeqSearch(A, n, nim)

if idx = -1 then

 output("Mahasiswa tidak ditemukan")

else

 input(kode)

 pos ← -1

 j ← 0

 while j < B[idx].Jml and pos = -1 do

 if B[idx].MK[j].Kode = kode then

 pos ← j

 endif

 j ← j + 1

 endwhile

 if pos = -1 then

 output("Mata kuliah tidak ditemukan")

 else

 for k ← pos to B[idx].Jml - 2 do

 B[idx].MK[k] ← B[idx].MK[k+1]

 endfor B[idx].Jml ← B[idx].Jml - 1

 output("Mata kuliah berhasil dihapus")

 endif

endif

endprocedure

procedure cariMahasiswaMK(in A : arrMhs, in B : arrKRS, in n : integer)

Kamus

Kode : string

i : integer

j : integer

ketemu : Boolean

Algoritma

output("Masukkan Kode MK : ")

input(kode)

for i ← 0 to n - 1 do

 ketemu ← false

 j ← 0

 while j < B[i].Jml and not ketemu do

 if B[i].MK[j].Kode = kode then

 output(A[i].NIM, A[i].Nama)

 ketemu ← true

 endif

 j ← j + 1

 endwhile

endfor

endprocedure

procedure cariMKMahasiswa(in A : arrMhs, in B : arrKRS, in n : integer)

Kamus

nim : string

idx : integer

j : integer

Algoritma

```
    Output("Masukkan NIM : ")
    input(nim)
    idx ← SeqSearch(A, n, nim)
    if idx = -1 then
        output("Mahasiswa tidak ditemukan")
    else
        for j ← 0 to B[idx].Jml - 1 do
            output(B[idx].MK[j].Kode, B[idx].MK[j].Nama, B[idx].MK[j].SKS)
        endfor
    endif
endprocedure
```

procedure selectionSortNilai(in/out A : arrMhs, in/out B : arrKRS, in n : integer, in desc : boolean)

Kamus

i : integer
j : integer
idx : integer
tempM : Mahasiswa
tempK : KRS

Algoritma

```
    for i ← 0 to n - 2 do
        idx ← i
        for j ← i + 1 to n - 1 do
            if desc then
                if rataNilai(B[j]) > rataNilai(B[idx]) then
                    idx ← j
                end if
            end if
        end for
    end for
```

```

endif
else
    if rataNilai(B[j]) < rataNilai(B[idx]) then
        idx ← j
    endif
endif
endif
endfor
tempM ← A[i]; A[i] ← A[idx]; A[idx] ← tempM
tempK ← B[i]; B[i] ← B[idx]; B[idx] ← tempK
endfor
endprocedure

```

procedure insertionSortSKS(in/out A : arrMhs, in/out B : arrKRS, in n : integer, in desc : boolean)

Kamus

```

i : integer
j : integer
tempM : Mahasiswa
tempK : KRS

```

Algoritma

```

for i ← 1 to n - 1 do
    tempM ← A[i]
    tempK ← B[i]
    j ← i
    if desc then
        while j > 0 and totalSKS(B[j-1]) < totalSKS(tempK) do
            A[j] ← A[j-1]; B[j] ← B[j-1]; j ← j - 1
        endwhile
    endif
endfor

```

```

        Else
            while j > 0 and totalSKS(B[j-1]) > totalSKS(tempK) do
                A[j] ← A[j-1]; B[j] ← B[j-1]; j ← j - 1
            endwhile
        Endif

        A[j] ← tempM
        B[j] ← tempK

    endfor
endprocedure

```

procedure tampilTranskrip(in A : arrMhs, in B : arrKRS, in n : integer)

Kamus

nim : string

idx : integer

i : integer

Algoritma

Output("Masukkan NIM : ")

input(nim)

idx ← BinarySearch(A, n, nim)

if idx = -1 then

 output("Mahasiswa tidak ditemukan")

else

 output(header transkrip: NIM, Nama)

 for i ← 0 to B[idx].Jml - 1 do

 output(B[idx].MK[i].Kode, B[idx].MK[i].Nama,
 B[idx].MK[i].SKS, B[idx].MK[i].Total, B[idx].MK[i].Grade)

 endfor

 output("Total SKS :", totalSKS(B[idx]))

 output("Rata-rata :", rataNilai(B[idx]))

```
endif  
endprocedure
```

Function main

KAMUS

Mhs : arrMhs

krs : arrKRS

n : integer

pilih : integer

ALGORITMA

n ← 0

pilih ← -1

while pilih ≠ 0 do

 output(menu: 0–12)

 input(pilih)

 if pilih = 1 then

 tambahMahasiswa(mhs, krs, n)

 else if pilih = 2 then

 editMahasiswa(mhs, n)

 else if pilih = 3 then

 hapusMahasiswa(mhs, krs, n)

 else if pilih = 4 then

 tambahMK(mhs, krs, n)

 else if pilih = 5 then

 editMK(mhs, krs, n)

 else if pilih = 6 then

 hapusMK(mhs, krs, n)

 else if pilih = 7 then

 cariMahasiswaMK(mhs, krs, n)

 else if pilih = 8 then

```
        cariMKMahasiswa(mhs, krs, n)
    else if pilih = 9 then
        selectionSortNilai(mhs, krs, n, true)
        tampilSemuaMahasiswa(mhs, krs, n)
    else if pilih = 10 then
        insertionSortSKS(mhs, krs, n, true)
        tampilSemuaMahasiswa(mhs, krs, n)
    else if pilih = 11 then
        tampilSemuaMahasiswa(mhs, krs, n)
    else if pilih = 12 then
        tampilTranskrip(mhs, krs, n)
    else if pilih = 0 then
        output("Program selesai")
    else
        output("Pilihan tidak tersedia")
    endif
endwhile
Endprogram
```